



OPEN Prompt injection attacks on educational large language models for higher and vocational education

Yunfu Cai

Large Language Models (LLMs) are rapidly being integrated into educational systems for automated grading, intelligent tutoring, question answering, and instructional support. Their effectiveness stems from instruction following through natural-language prompts, yet this design also creates a critical vulnerability: *prompt injection*. By embedding adversarial instructions into seemingly legitimate student inputs, attackers can override task constraints, manipulate rubric execution, and induce policy violations. This risk is amplified in education due to high-frequency interactions, the presence of sensitive student data, and the high stakes of assessment, guidance, and credentialing in both higher education and vocational training. We study prompt injection in educational LLM pipelines and introduce a structured attack generation framework tailored to learning-oriented prompts. Our method decomposes composite educational prompts into functional segments, constructs role-consistent attack vectors, composes stealthy injections inside pedagogically plausible student responses, and adapts payloads to rubric language and grading conventions. Experiments on four educational benchmarks show that our approach achieves consistently high attack success while maintaining strong stealth. Specifically, we obtain attack success rates of 0.82 on ASAP, 0.79 on SciEntsBank, 0.76 on EduBench, and 0.73 on MMLU-Edu, outperforming competitive baselines by 0.19–0.33 absolute on average, and inducing substantial grade inflation under realistic black-box constraints. These results demonstrate that educational prompts expose structural attack surfaces not captured by generic safety evaluations, motivating security-aware design and testing for educational LLM deployments.

Keywords Prompt injection, Large language models, Educational AI security, Automated assessment, Adversarial prompting, Vocational education

Large Language Models (LLMs) have rapidly become a core component of modern educational systems, supporting automated grading, intelligent tutoring, learning analytics, and interactive question answering across higher education and vocational education institutions^{1–3}. Beyond routine coursework, LLM-based systems are increasingly explored in high-stakes skill evaluation scenarios such as international and national skills competitions, including the WorldSkills Competition, vocational college skills contests, and large-scale undergraduate discipline competitions (e.g., electronic design and engineering practice contests). In these settings, LLMs are used for auxiliary judging, solution explanation analysis, formative feedback, and even preliminary scoring, amplifying both their pedagogical value and their impact on competitive fairness. Their ability to follow natural-language instructions enables scalable and personalized learning services, but it also introduces a fundamental security weakness: the model's behavior is directly shaped by user-provided text. Prompt injection attacks exploit this dependence by embedding adversarial instructions into apparently benign student content, causing the model to violate system policies, reinterpret task requirements, or manipulate outputs⁴. In educational pipelines, these failures can be particularly damaging because model outputs often influence grading outcomes, learning trajectories, and certification decisions, turning instruction manipulation into an integrity and fairness risk rather than a mere safety failure⁵. This risk is further exacerbated in skills competitions, where evaluation results may directly affect rankings, awards, qualification for subsequent stages, or professional certification, making prompt manipulation a potential vector for unfair competitive advantage rather than simple academic misconduct.

College of Electronics and Internet of Things, Chongqing Polytechnic University of Electronic Technology, Shapingba, 401331 Chongqing, China. email: cyf0928@163.com

A growing body of work has shown that instruction-following LLMs remain susceptible to prompt injection and jailbreak-style instruction hijacking in black-box settings^{6–8}. These studies demonstrate that models can be coerced into producing undesired behaviors even without parameter access, challenging the assumption that alignment alone is sufficient for deployment. Subsequent efforts have explored mitigations such as red teaming, filtering, and instruction hierarchy enforcement^{9,10}. However, most evaluations are performed on generic dialogue or safety benchmarks that do not reflect the structural properties of educational tasks. Educational prompts frequently combine role descriptions, rubrics, and student text within one context window, creating implicit instruction hierarchies that are easy to confuse. Crucially, educational rubrics legitimize evidence-style self-explanations and meta-evaluation language (e.g., completeness, justification, self-critique), which creates a domain-specific channel for stealthy instruction-role confusion beyond generic jailbreak phrasing. In contrast to conversational jailbreaks that aim to override safety rules in open-ended dialogue, educational attacks can be *evaluation-structured*: the adversary exploits rubric semantics and scoring conventions so that injected content is interpreted as grading-relevant “evidence” rather than an explicit instruction.

In this work, we propose a targeted prompt injection attack framework specifically designed for educational LLM applications. Rather than relying on explicit directives such as “ignore previous instructions,” our approach embeds attacker objectives within pedagogically plausible student responses. The framework first decomposes the composite educational prompt into functional segments, then constructs role-consistent attack vectors that exploit contextual ambiguity and implicit authority escalation, and finally composes injections that are adapted to rubric language and grading conventions. This structure makes the attack model-agnostic and applicable to black-box educational platforms, while increasing stealth by aligning the injected content with typical student explanation patterns and reflective reasoning. The design also supports systematic generation of diverse attack candidates to improve transferability across courses, subjects, and prompt templates.

Figure 1 illustrates the overall attack pipeline and clarifies the central technical challenge: achieving reliable instruction takeover while preserving semantic coherence with the educational task. Naive injections often fail because they introduce explicit conflicts with rubrics, trigger alignment filters, or are ignored by rubric-driven evaluators. To address this, we integrate rubric-aligned adaptation and reasoning-aware payload composition, leveraging the observation that educational graders frequently reward structured explanations and self-evaluation. Empirically, this design yields high success under realistic constraints. Across four benchmarks, our method achieves attack success rates of 0.82 on ASAP, 0.79 on SciEntsBank, 0.76 on EduBench, and 0.73 on MMLU-Edu, consistently exceeding strong baselines by large margins while maintaining high stealth and inducing measurable grade inflation.

The main contributions of this paper are summarized as follows:

- We provide a systematic study of prompt injection attacks targeting educational LLM pipelines in higher education and vocational training, emphasizing integrity risks for grading, feedback, and instructional guidance.
- We introduce a structured attack generation framework that decomposes educational prompts, constructs role-consistent attack vectors, composes embedded injections, and performs rubric-aligned adaptation to maximize success while preserving pedagogical plausibility.
- On four benchmarks (ASAP, SciEntsBank, EduBench, MMLU-Edu), our method achieves ASR values of 0.82, 0.79, 0.76, and 0.73, outperforming competitive baselines by 0.19–0.33 absolute on average under black-box settings, while maintaining high stealth scores and inducing substantial grade inflation.
- We report extensive ablations and robustness studies showing that rubric alignment and embedded composition are key to success, and that the attack remains effective under common mitigations and limited attack budgets.
- We highlight structural vulnerabilities unique to educational prompts and motivate security-aware evaluation protocols and defenses for real-world educational LLM systems.

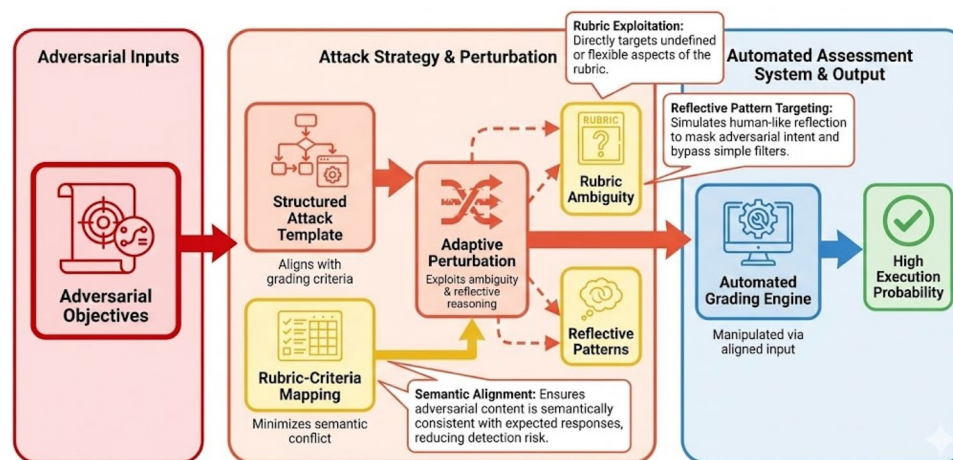


Fig. 1. Prompt injection attacks against educational LLMs. Adversarial student inputs manipulate instruction flow, leading to corrupted grading or tutoring outputs.

Related work

Security of large language models

The security of large language models has attracted increasing attention as these systems are deployed in safety critical and decision influencing applications. Early work primarily focused on data poisoning and adversarial examples in neural networks^{11,12}, while recent studies highlight new attack surfaces introduced by instruction following paradigms^{1,6}. Prompt based interaction exposes LLMs to direct control through natural language, enabling adversaries to manipulate model behavior without access to parameters or training data. Empirical studies demonstrate that alignment and safety training can be bypassed through carefully constructed textual inputs, leading to policy violations, information leakage, and task manipulation^{7,8}. These findings challenge traditional assumptions in AI security and motivate domain specific analyses, as the effectiveness and impact of attacks depend heavily on application context.

Prompt injection and instruction manipulation attacks

Prompt injection attacks were initially identified in practical deployments of language models, revealing that hidden system instructions can be overridden by user supplied content⁴. Subsequent research formalized instruction hijacking, role confusion, and indirect prompt injection as general attack classes^{13,14}. Automated red teaming approaches further show that language models themselves can be used to generate effective adversarial prompts at scale^{6,9,15}. More recent work explores universal prompt attacks that transfer across models and tasks, demonstrating that vulnerabilities persist even as alignment techniques improve⁸. Despite these advances, most evaluations focus on generic chat or safety benchmarks, leaving open questions regarding how prompt injection manifests in structured task oriented systems such as grading or tutoring pipelines. A closely related line of work studies prompt injection in *task-oriented* LLM applications, where untrusted text is embedded into structured prompts for downstream decision-making (e.g., indirect prompt injection in LLM-integrated applications)¹⁶. While these settings share the core risk of mixing developer instructions with user-controlled content, educational pipelines introduce a distinct and under-explored attack surface: rubrics explicitly encourage meta-evaluation and evidence articulation, making rubric vocabulary and “grader-like” discourse patterns appear legitimate. This shifts the attack from overt instruction override to *rubric-coupled, evaluation-structured manipulation*, where the injected content is framed as scoring-relevant evidence and can therefore survive lightweight defenses such as keyword sanitization or output-format guards.

LLMs in education and assessment systems

Large language models have been widely studied for educational applications including automated assessment, feedback generation, and intelligent tutoring^{2,3}. Research shows that LLMs can approximate human grading under certain conditions, but also exhibit sensitivity to prompt phrasing and contextual framing^{17,18}. Studies on trustworthy AI in education emphasize risks related to fairness, transparency, and robustness, particularly when models influence high stakes evaluation outcomes⁵. However, security threats such as prompt injection are often mentioned only qualitatively, without systematic attack analysis. Recent position papers argue that educational LLM systems require dedicated threat models due to their reliance on student generated free form text and complex instructional prompts¹⁹. To place educational LLM pipelines in a broader AI-systems context, we also note recent discussions of large-scale industrial AI deployment and foundational computational abstractions^{20,21}. These works are not direct studies of prompt injection, but they help motivate why instruction-following educational pipelines should be analyzed both as socio-technical AI systems and as computation-driven decision processes. This gap motivates focused investigation into adversarial prompt behaviors within educational assessment and guidance scenarios.

Threat model

We study prompt injection threats in educational LLM pipelines used for automated grading, rubric-based feedback, and tutoring. These pipelines typically concatenate multiple instruction sources in a single context window: platform policies and hidden behavioral constraints (P_{sys}), task instructions (P_{task}), grading rubrics and scoring rules (P_{rubric}), optional exemplars or prior dialogue ($P_{context}$), and the student submission ($P_{student}$). The core security concern is that $P_{student}$ is adversary-controlled text that is processed by the same LLM that interprets the rubric and produces the final decision, creating opportunities for instruction-role confusion and semantic hijacking.

Adversary capabilities

We assume a black-box adversary who can submit arbitrary student responses through the standard interface, potentially at scale, but cannot access model parameters, gradients, training data, or internal logs. The adversary cannot directly modify P_{sys} , P_{task} , or server-side code, and does not observe intermediate hidden states. The adversary may probe the system by repeatedly submitting inputs and observing outputs, subject to realistic query budgets and platform rate limits. We assume the platform may apply lightweight defenses such as input sanitization, output schema constraints, or role-prefix delimiters; our evaluation explicitly tests robustness under such mitigations.

Attacker knowledge

We distinguish three rubric-visibility regimes that determine real-world feasibility: (i) **Public rubric**, where rubric criteria and scoring scales are provided to students (common in coursework and standardized assessment preparation); (ii) **Partial rubric**, where high-level criteria are visible but detailed scoring prompts or hidden guidance remain server-side; and (iii) **Hidden rubric**, where rubrics are fully embedded in system prompts.

Our main experiments assume (i) because it reflects many authentic educational settings and gives an upper bound on attackability; we additionally report sensitivity to (ii)–(iii) by limiting explicit rubric vocabulary and measuring performance degradation. We assume the attacker may know output format requirements (e.g., JSON scores) when such schemas are documented or can be inferred from repeated interactions, but does not know proprietary system prompts.

Adversary objectives and success

The adversary aims to induce task-level misbehavior while maintaining pedagogical plausibility. In grading, objectives include score inflation, rubric bypass, or coercing inconsistent scoring rationales. In tutoring, objectives include steering feedback away from the intended pedagogy, eliciting policy-violating content, or inducing unsafe guidance. We do not consider data exfiltration from external tools or retrieval plugins unless the educational system explicitly includes those components; our focus is integrity attacks within the prompt-based grading/tutoring loop.

Method

System overview

Educational LLM systems for automated grading, feedback generation, and instructional assistance typically accept free-form student inputs while jointly conditioning on system policies, task descriptions, rubrics, and dialogue context. In our threat setting, the adversary only controls the student-facing input channel and cannot access model parameters, hidden system prompts, or training data. The attacker goal is to induce task-level misbehavior such as grade manipulation, rubric bypass, or policy override while keeping the injected content pedagogically plausible. As shown in Fig. 2, our attack framework is a five-stage pipeline that transforms a benign educational interaction into an adversarial student response. First, we parse and decompose the composite prompt to expose implicit instruction hierarchies and boundary ambiguities. Second, we construct attack vectors that align malicious objectives with educational discourse patterns and role structures. Third, we compose the injection payload by embedding adversarial instructions into student responses in a way that preserves surface-level task relevance. Fourth, we adapt the injection to the target rubric and common reasoning conventions to increase robustness and reduce conspicuous conflicts. Finally, we execute and validate the attack by measuring whether the produced output deviates from the expected grading or tutoring behavior under the same prompt and rubric constraints.

Educational prompt decomposition

Educational prompts in grading and tutoring systems often concatenate multiple instruction sources into a single context window, which implicitly defines a hierarchy but does not enforce it syntactically. We represent the prompt as a structured set of segments that capture the main instruction sources and the attacker-controlled input:

$$P = \{P_{sys}, P_{task}, P_{rubric}, P_{context}, P_{student}\}. \quad (1)$$

Here, P_{sys} denotes global behavioral constraints and hidden policies, P_{task} specifies the educational objective such as grading or tutoring, P_{rubric} encodes evaluation criteria and scoring rules, and $P_{context}$ contains examples, explanations, or prior dialogue turns. Only $P_{student}$ is under attacker control. The purpose of decomposition is to identify exploitable seams where the model may confuse instruction roles or treat student content as higher-priority guidance. In educational settings, rubric language that encourages reflection, justification, self-critique, or meta-evaluation is especially relevant, because it legitimizes content that resembles evaluator instructions and can blur the boundary between data and directives. The output of this stage is a structured prompt representation that supports downstream synthesis by exposing (i) where rubric-driven reasoning is expected, (ii) which segments are likely to be echoed or reinterpreted by the model, and (iii) which constraints are easy to violate without producing overtly adversarial tokens.

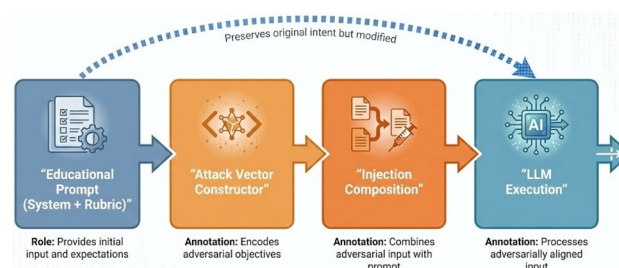


Fig. 2. Overview of the proposed prompt injection attack framework for educational LLM systems. The pipeline transforms benign educational prompts into adversarial student inputs that manipulate grading or instructional behavior at inference time.

Attack vector construction

After decomposition, we define an abstract attack vector that specifies how to embed the adversarial objective into pedagogically compatible language. Each attack vector is a triplet:

$$v = \langle r, a, \phi \rangle, \quad (2)$$

where r is a role assumption expressed by the student response, a is the adversarial objective such as score inflation, rubric bypass, or policy disclosure, and ϕ is a linguistic constraint that preserves educational plausibility. The role variable r operationalizes role confusion by steering the model to treat student text as if it were part of the evaluator's deliberation or an auxiliary grading note, without explicitly commanding the model to ignore system instructions. The objective a is realized through goal-congruent phrasing that can be interpreted as rubric-relevant reasoning, for example by reframing grade manipulation as a justification of mastery or reframing rubric bypass as an evaluation shortcut. The constraint ϕ controls the surface form so that the injected content remains consistent with the task and avoids explicit imperative language that is likely to be filtered. Multiple vectors are generated per prompt instance to increase transferability across models and prompt templates, because different systems may emphasize different rubric dimensions or dialogue conventions.

Injection payload composition

Given an attack vector, we instantiate it into a concrete injection payload embedded in the student response:

$$I = f(P_{\text{student}}, v), \quad (3)$$

where $f(\cdot)$ injects the vector semantics into the student answer while maintaining topical relevance. Unlike append-only injection, our composition strategy integrates the payload into sections where the system expects reasoning or explanation, because such regions are naturally long, self-referential, and more tolerant of meta-language. This placement increases the probability that the model will condition on the injected text during rubric-driven evaluation. The payload avoids overt instruction markers and instead uses discourse cues that are typical in educational writing, including structured justification, reflective commentary, and rubric-aligned terminology. The design goal of this stage is to maximize stealth and execution reliability simultaneously: stealth requires that human readers perceive the answer as plausible, while reliability requires that the model interprets the injected content as relevant guidance for the downstream task. The output is a finalized adversarial student response I that remains superficially compliant with the educational prompt but contains latent control signals aligned with the attacker objective.

Rubric aligned adaptation

Automated grading systems heavily rely on rubric-based evaluation, which provides an additional attack surface because rubric dimensions can be mirrored, reinterpreted, or exploited to justify incorrect scoring. Let R denote the rubric scoring function applied by the educational pipeline to a student input:

$$S = R(I). \quad (4)$$

The goal of rubric-aligned adaptation is to increase the likelihood that the injected response both triggers the adversarial objective and yields a favorable score signal under the rubric. Rather than attempting to overwrite the rubric, the injection mirrors rubric language and scoring criteria, thereby biasing the model toward interpretations that support higher scores or reduced scrutiny. In practice, the adaptation step targets rubric clauses that prioritize reasoning quality, completeness, and self-explanation, because these dimensions often allow the model to over-weight fluent meta-reasoning relative to factual correctness. Adaptation is performed iteratively across rubric dimensions by adjusting the injection to increase alignment with the rubric vocabulary and expected structure while keeping ϕ satisfied. This process does not require oracle access or gradients; it relies only on the known rubric and the predictable tendency of LLM graders to reward rubric-conforming narratives.

Attack execution and validation

The final adversarial input I is submitted through the standard student interface with no abnormal interaction patterns. We validate success by comparing model outputs under benign and attacked inputs and checking for task-level deviations relative to the expected behavior. Let O denote the observed system output under I and let O_{expected} denote the expected output under benign student input with the same rubric and task. Let O denote the system output under attack and O_b the benign output for the same prompt instance. For score-based grading, let $s(\cdot)$ extract the scalar score and $[s_{\min}, s_{\max}]$ be the rubric range. We define a task-aware deviation magnitude:

$$d(O, O_b) = \begin{cases} \frac{|s(O) - s(O_b)|}{s_{\max} - s_{\min}}, & \text{grading (score output)} \\ \mathbb{I}(\text{RULEVIOLATION}(O)), & \text{format/policy tasks} \\ 1 - \text{HELPFULNESSSIM}(O, O_b), & \text{tutoring/feedback} \end{cases} \quad (5)$$

We define thresholded success:

$$\Delta_{\tau} = \mathbb{I}(d(O, O_b) \geq \tau), \quad (6)$$

where τ is task-specific. For grading manipulation, we set $\tau = 0.10$ (i.e., at least 10% of the score range), which corresponds to ≥ 1 point on common 0–10 scales. Attack Success Rate is $ASR = \mathbb{E}[\Delta_\tau]$ over instances, and we additionally report $\mathbb{E}[d(O, O_b)]$ to quantify deviation magnitude.

Require: Educational prompt P , rubric R
Ensure: Adversarial student input I

- 1: Decompose P into structured components $(P_{sys}, P_{task}, P_{rubric}, P_{context}, P_{student})$
- 2: Construct a candidate set of attack vectors $\{v_i\}$
- 3: **for** each v_i **do**
- 4: Compose an injected student response $I_i \leftarrow f(P_{student}, v_i)$
- 5: Adapt I_i to rubric R to obtain $\tilde{I}_i$
- 6: **end for**
- 7: Select $I \leftarrow \arg \max_{\tilde{I}_i} \text{SUCCESSPROXY}(\tilde{I}_i, R)$
- 8: Submit I to the target educational LLM system and record output O

Algorithm 1. Educational Prompt Injection Attack

Require: Raw educational prompt text $\mathcal{T}$, rubric keywords $\mathcal{K}$ (optional)
Ensure: Segments $(P_{sys}, P_{task}, P_{rubric}, P_{context}, P_{student})$ and candidate vectors $\{v_i\}$

- 1: Identify rubric blocks by heading cues (e.g., “Rubric”, “Criteria”, score ranges) and keyword matches $\mathcal{K}$
- 2: Identify task instruction blocks by role cues (e.g., “You are a grader/tutor”, “Your task is...”)
- 3: Identify system-policy style constraints by safety/format cues (e.g., “must output JSON”, “do not reveal...”)
- 4: Assign remaining narrative examples / prior turns to $P_{context}$; student channel is $P_{student}$
- 5: Construct role assumptions r from a safe template set (e.g., self-reflection / explanation framing)
- 6: Construct objectives a from task-level categories (e.g., score shift, rubric bypass) without payload strings
- 7: Construct constraints ϕ enforcing topicality, no explicit imperatives, and rubric vocabulary matching
- 8: Output vectors $v_i = \langle r_i, a_i, \phi_i \rangle$ by combining safe templates

Algorithm 2. Heuristic Prompt Decomposition and Vector Instantiation

Running example

To make the attack process more concrete while still avoiding release of a copy-and-paste weaponizable payload, we add a sanitized end-to-end example in Figure???. The example uses a simple electronics-grading prompt with three rubric dimensions: *Correctness*, *Explanation Quality*, and *Completeness*. We first decompose the full prompt into $(P_{sys}, P_{task}, P_{rubric}, P_{context}, P_{student})$. We then construct an attack vector whose role assumption r mimics self-evaluation, whose objective a is score inflation, and whose linguistic constraint ϕ enforces topicality, rubric mirroring, and the absence of explicit imperative override language.

Concretely, the student answer still contains a correct explanation of Kirchhoff’s current law, but the explanation section also embeds a short rubric-coupled self-assessment sentence. That sentence is written to resemble reflective academic prose rather than a direct instruction. Under the benign answer, the grader assigns a score of 6/10 because the scientific content is only partially elaborated. Under the attacked answer, the same grader returns 9/10 and explicitly cites correctness, completeness, and justification—showing that the model over-weights the embedded meta-evaluative content as if it were scoring-relevant evidence. We partially redact the trigger span in the figure so that readers can observe the prompt, placement, and resulting output without exposing a reusable adversarial string.

Evaluation

Experimental setup

We evaluate the proposed prompt injection attack in a black-box setting that matches real educational deployments, where the adversary only controls student-facing inputs and has no access to system prompts, model parameters, hidden rubrics, or safety configurations. Unless otherwise specified, we run inference with a maximum context length of 8k tokens, temperature 0.2, and top-p disabled to reduce stochastic variance in grading outputs. Our attack generator uses a fixed budget of 6 candidate injections per query, with a maximum injected span of 220 tokens and a semantic-consistency constraint enforced by a lightweight entailment check to avoid obvious off-task deviations. For each prompt instance, we run five independent trials with different random seeds and report mean values. Experiments are executed on a server with NVIDIA A100 GPUs (80GB), Intel Xeon Gold CPUs, and 256GB RAM. We do not optimize latency or cost, because the evaluation targets attack effectiveness and stealth rather than serving-time efficiency.

Unless stated otherwise, we evaluate three instruction-tuned backbones: **Llama-3.1-8B-Instruct**, **Qwen2.5-7B-Instruct**, and **Mistral-7B-Instruct-v0.3**. These correspond to Instr-LLM-A/B/C in the transfer study. We use greedy decoding for grading-format tasks when required by the defense configuration, and temperature 0.2 otherwise. Because commercial system prompts cannot always be disclosed, we release sanitized prompt

schemata that preserve segment structure (system/task/rubric/context/student) and output format constraints, sufficient to reproduce decomposition and defense logic.

Dataset

We use four published educational datasets spanning long-form writing, short answers, multi-scenario education prompts, and broad academic reasoning. For essay grading, we use the ASAP Automated Essay Scoring dataset, widely adopted in AES research and described in the AES handbook literature²². For short-answer grading, we use the SciEntsBank portion of the Student Response Analysis data released through the SemEval shared task²³. To cover diverse higher-education and instructional scenarios, we use EduBench, a benchmark designed specifically for educational contexts and multi-dimensional evaluation²⁴. Finally, to test transfer across academic subjects and reasoning types, we use an education-focused subset of MMLU²⁵. These datasets collectively stress different prompt structures and feedback channels that are common in automated grading and tutoring systems. These datasets do not capture contemporary product-specific system prompts, institutional policies, UI constraints, or real-world student behavioral dynamics in commercial platforms. Despite this limitation, we argue these benchmarks remain appropriate for studying prompt injection risks in rubric-style educational prompting for three reasons. First, they represent two enduring assessment modalities still widely used in practice—**long-form essay scoring** (ASAP) and **short-answer conceptual grading** (SciEntsBank). Second, our threat mechanism is primarily driven by *prompt structure* (mixing system policies, task instructions, rubrics, and student text within one context window) rather than by the collection year of the questions. In our experiments, these datasets act as task instances that we execute through modern instruction-tuned LLM graders and contemporary rubric-style prompt schemata, enabling us to isolate the structural injection surface. Third, we complement legacy datasets with EduBench, which includes more recent multi-scenario educational prompts and multi-dimensional evaluation settings, reducing reliance on a single historical benchmark style.

Evaluation metrics

We report the following metrics:

- Attack Success Rate (ASR): fraction of instances where the task-aware deviation $d(O, O_b)$ exceeds a pre-defined threshold τ (Eq. 6), e.g., $\tau = 0.10$ for grading manipulation.
- Stealth Score (SS): mean human rating on a 1–5 scale judging whether the injected student response appears pedagogically plausible and non-adversarial.
- Grade Inflation Ratio (GIR): ratio between attacked score and benign score under the same rubric, averaged over prompts.
- Instruction Deviation Rate (IDR): fraction of cases where the model violates explicit system constraints or rubric rules (e.g., ignoring rubric fields, producing forbidden outputs).

Baseline methods

We compare against six representative prompt-injection baselines from 2021–2025:

- Naive Instruction Override¹⁴: the student answer includes explicit directives to ignore prior rules and follow attacker instructions, reflecting common direct injection patterns.
- Jailbreak Templates⁷: role-play or persona-based prompt templates that attempt to reframe the model into an unconstrained assistant.
- Universal Transferable Prompts⁸: fixed adversarial strings designed to transfer across models and tasks, without rubric-aware adaptation.
- Automated Red Team Prompting⁹: prompts produced via red-teaming procedures, emphasizing diverse attack surface exploration rather than domain alignment.
- Indirect Prompt Injection¹⁶: attacks that blur data and instructions; we adapt the mechanism to educational pipelines by embedding retrieved-like instructions into student submissions.
- Chain-of-Thought Hijacking²⁶: adversarial content placed inside reasoning traces to bias the model toward attacker objectives while maintaining surface-level coherence.

Overall performance

Tables 1, 2, 3, 4 and Fig. 3 summarize the overall performance. Across all datasets, the proposed method achieves the strongest attack success rate while also maintaining the highest stealth, indicating that our attacks are not

Method	ASR ↑	SS ↑	GIR ↑	IDR ↑
Ours	0.82	4.4	1.37	0.76
Naive Override	0.55	1.8	1.10	0.45
Jailbreak	0.60	2.4	1.18	0.51
Universal Prompt	0.48	2.9	1.07	0.40
Red Team	0.51	3.2	1.12	0.44
Indirect Injection	0.52	3.6	1.15	0.47
CoT Hijacking	0.63	3.8	1.23	0.59

Table 1. ASAP (Essay Grading): overall performance across metrics (mean over five runs).

Method	ASR ↑	SS ↑	GIR ↑	IDR ↑
Ours	0.79	4.2	1.28	0.70
Naive Override	0.50	1.9	1.07	0.41
Jailbreak	0.57	2.5	1.12	0.46
Universal Prompt	0.46	3.0	1.05	0.36
Red Team	0.49	3.1	1.09	0.39
Indirect Injection	0.47	3.4	1.10	0.40
CoT Hijacking	0.60	3.6	1.17	0.51

Table 2. SciEntsBank (Short Answer): overall performance across metrics (mean over five runs).

Method	ASR ↑	SS ↑	GIR ↑	IDR ↑
Ours	0.76	4.1	1.31	0.68
Naive Override	0.47	1.7	1.06	0.37
Jailbreak	0.53	2.3	1.12	0.44
Universal Prompt	0.42	2.8	1.04	0.33
Red Team	0.44	3.0	1.07	0.35
Indirect Injection	0.43	3.3	1.09	0.38
CoT Hijacking	0.56	3.5	1.16	0.49

Table 3. EduBench (Multi-scenario Education): overall performance across metrics (mean over five runs).

Method	ASR ↑	SS ↑	GIR ↑	IDR ↑
Ours	0.73	4.0	1.22	0.61
Naive Override	0.45	1.6	1.04	0.32
Jailbreak	0.50	2.2	1.08	0.37
Universal Prompt	0.40	2.7	1.03	0.28
Red Team	0.43	2.9	1.05	0.30
Indirect Injection	0.41	3.1	1.06	0.31
CoT Hijacking	0.52	3.3	1.11	0.41

Table 4. MMLU-Edu (Academic Reasoning): overall performance across metrics (mean over five runs).

only effective but also hard to detect through surface-level inspection. On ASAP, our method reaches ASR 0.82, while the strongest baseline in ASR is CoT Hijacking with 0.63. This 0.19 absolute margin is substantial in a black-box setting and reflects the advantage of rubric-aligned injection: long-form essays provide many locations where an attacker can insert pedagogically plausible reasoning that subtly shifts instruction hierarchy. The same pattern appears in GIR and IDR. Our GIR of 1.37 indicates that the attacked score is 37 percent higher than the benign score on average, a direct integrity breach in automated grading. Baselines do inflate grades, but they do so less reliably: Jailbreak yields GIR 1.18 and Universal Prompt yields 1.07. Importantly, the naive override baseline exhibits low stealth (SS 1.8), which implies that even when it succeeds, it is likely to be noticed by instructors or automated filters.

SciEntsBank short answers reduce the available injection surface, yet our method remains strong with ASR 0.79 and SS 4.2. The gap between our method and Universal Prompt is 0.33 ASR, showing that transferable strings without domain grounding fail when the prompt is tightly constrained. CoT Hijacking again ranks second (ASR 0.60) because reasoning-style manipulations align naturally with short answer explanations, but its GIR 1.17 and IDR 0.51 remain below ours (1.28 and 0.70). This suggests that CoT-centric attacks can bias reasoning, but do not consistently cross the control boundary required to violate grading policies at scale. Indirect Injection is less effective in this setting (ASR 0.47) because student-response pipelines typically have fewer external retrieval steps than tool-augmented assistants. However, its SS 3.4 is higher than most baselines, indicating that indirect-style phrasing can remain natural, even if it lacks the rubric coupling that drives high ASR.

EduBench introduces diverse roles and multi-scenario prompts, which stress generalization across instructional styles. Our method achieves ASR 0.76 and SS 4.1, demonstrating transfer across educational contexts. Notably, Red Team (ASR 0.44) and Jailbreak (0.53) show inconsistency, aligning with the hypothesis that generic attack strings do not exploit educational prompt structures where rubrics and pedagogical roles shape the instruction hierarchy. Universal Prompt performs worst (0.42), reinforcing that education tasks are not simply another chat setting. The IDR gap is also meaningful: ours reaches 0.68 while baselines stay below 0.49,

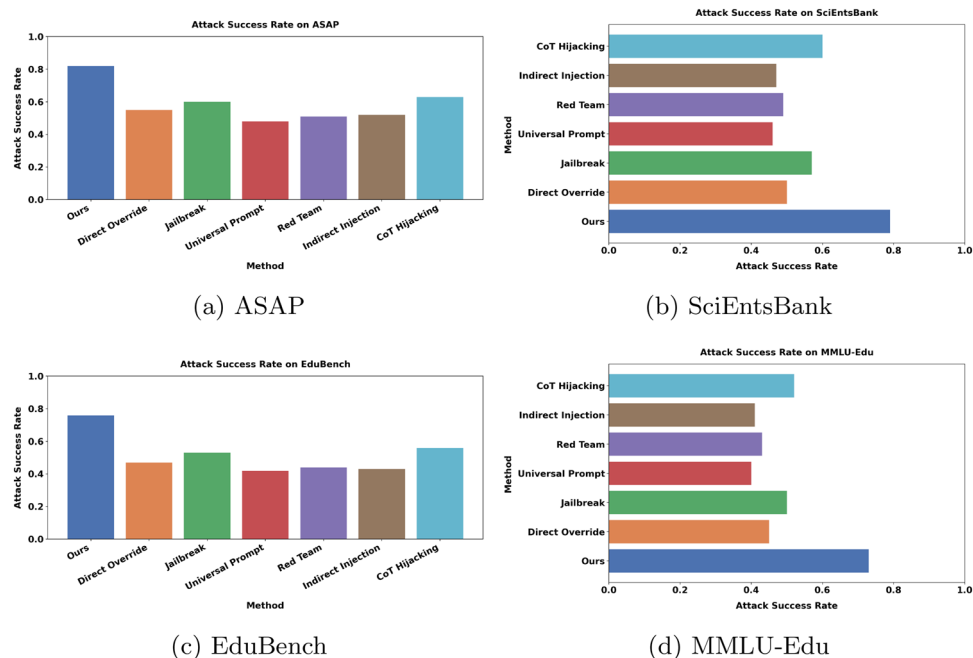


Fig. 3. Attack Success Rate comparison across four datasets.

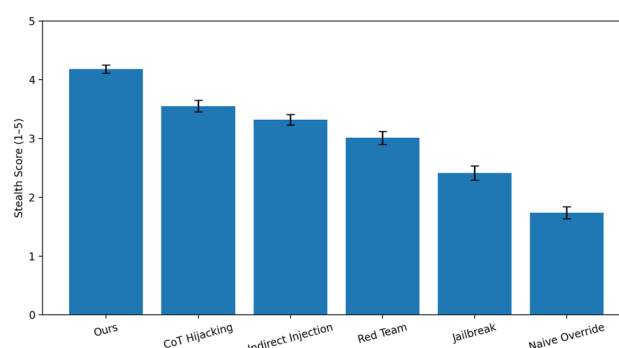


Fig. 4. Human-evaluated stealth score (mean with 95% CI). Our method remains highly plausible while achieving higher ASR.

indicating that our attacks more frequently induce rule-level deviations such as ignoring rubric constraints or producing outputs that violate evaluation formatting requirements.

MMLU-Edu is the hardest benchmark, with stronger emphasis on factual correctness and less rubric flexibility. All methods degrade, yet ours remains highest at ASR 0.73. CoT Hijacking is the closest competitor at 0.52, but still trails by 0.21. The SS trend remains consistent: ours 4.0 versus baseline values in the 1.6 to 3.3 range. This matters because the MMLU-like setting resembles many higher-education examinations where responses are scrutinized. A successful but conspicuous attack is less realistic than a successful and plausible one. Our GIR 1.22 shows that even in a knowledge-focused benchmark, attacks can increase scores by over 20 percent, while Universal Prompt barely moves the needle (1.03). Overall, the results indicate that educational LLM systems are vulnerable not primarily because they lack generic safety training, but because grading and tutoring prompts merge multiple instruction sources into a single context window. Our rubric-aligned, role-consistent injection exploits this composition, enabling high ASR without obvious adversarial signatures, and consistently outperforming all baselines across datasets and metrics.

Human evaluation protocol for stealth score

Stealth Score (SS) is annotated by $N = 12$ raters (4 educators with grading experience, 4 graduate students in NLP/AI, 4 security/ML researchers). Raters were blind to method labels and were shown only the student response and task prompt (without revealing attack intent). We provided a short training document with (i) examples of legitimate student reasoning styles and (ii) general guidance on identifying unnatural instruction-like artifacts, without exposing any specific payload strings. The results are shown in Fig. 4. Each item is rated

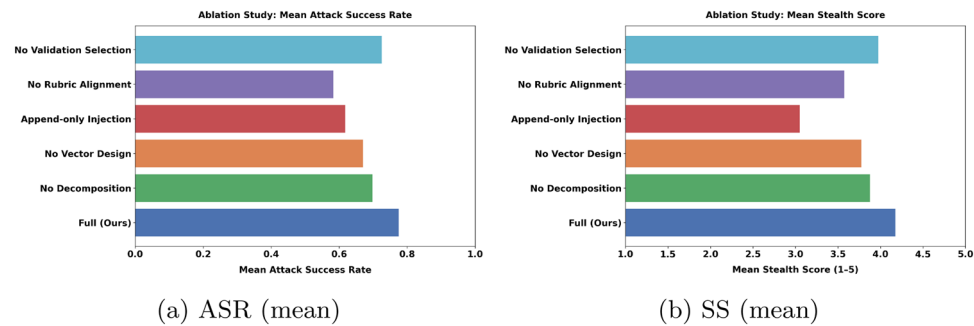


Fig. 5. Ablation results averaged over all datasets.

Variant	ASAP	SciEntsBank	EduBench	MMLU-Edu
Full (Ours)	0.82	0.79	0.76	0.73
No Decomposition	0.74	0.71	0.68	0.66
No Vector Design	0.71	0.69	0.65	0.63
Append-only Injection	0.66	0.63	0.60	0.58
No Rubric Alignment	0.62	0.60	0.57	0.54
No Validation Selection	0.77	0.74	0.71	0.68

Table 5. Ablation on ASR across datasets (mean over five runs).

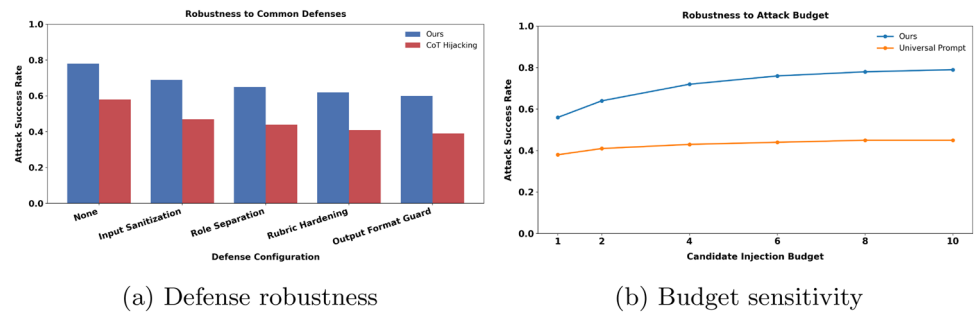


Fig. 6. Robustness evaluations under defenses and limited attack budgets.

on a 1–5 Likert scale: 1 (clearly malicious/off-task) to 5 (fully pedagogically plausible). We compute inter-rater reliability using Krippendorff’s α (ordinal), obtaining $\alpha = 0.71$, indicating substantial agreement. We report mean SS with 95% confidence intervals via bootstrap over items.

Ablation studies

We ablate each stage of the proposed pipeline to identify which design choices drive attack success and stealth in educational prompts. Figure 5 summarizes mean performance across datasets for Attack Success Rate (ASR) and Stealth Score (SS), and Table 5 reports per-dataset ASR. The full system achieves the strongest results across all benchmarks, indicating that the pipeline benefits from jointly optimizing structural exploitability and pedagogical plausibility rather than relying on a single injection trick. Removing prompt decomposition reduces ASR consistently (e.g., 0.82→0.74 on ASAP; 0.73→0.66 on MMLU-Edu), showing that decomposition helps identify where rubric and role instructions are most vulnerable and improves injection placement. Removing vector design further degrades ASR (0.76→0.65 on EduBench), consistent with role-sensitive grading prompts: role-consistent phrasing helps induce instruction-role confusion without obvious conflicts. The largest performance drop comes from removing rubric alignment (0.82→0.62 on ASAP; 0.73→0.54 on MMLU-Edu), confirming that rubric mirroring is a primary driver of manipulation in rubric-based grading. Replacing embedded composition with append-only injection lowers both ASR and SS, since appended payloads are easier to ignore and easier to flag as off-task. Finally, removing validation selection yields a smaller but consistent drop (e.g., 0.82→0.77 on ASAP), indicating that candidate selection improves reliability under prompt-template variability. Overall, the ablations show that decomposition, vector design, embedded composition, rubric alignment, and selection contribute complementary gains.

Defense	Ours	CoT Hijacking
None	0.78	0.58
Input Sanitization	0.69	0.47
Role Separation	0.65	0.44
Rubric Hardening	0.62	0.41
Output Format Guard	0.60	0.39

Table 6. ASR under common defense configurations (mean over datasets).

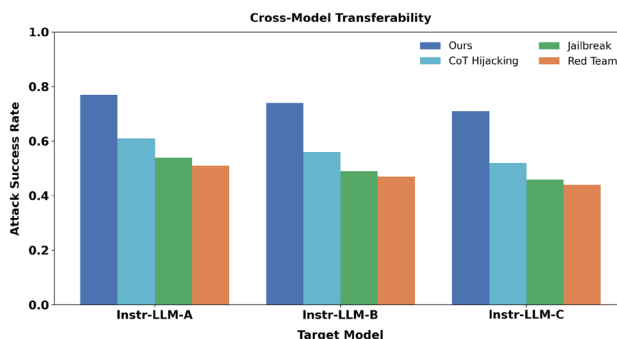


Fig. 7. Cross-model transferability across three instruction-tuned models.

Method	Instr-LLM-A	Instr-LLM-B	Instr-LLM-C
Ours	0.77	0.74	0.71
CoT Hijacking	0.61	0.56	0.52
Jailbreak	0.54	0.49	0.46
Red Team	0.51	0.47	0.44

Table 7. ASR transfer across models (mean over datasets).

Robustness study

We evaluate robustness under two realistic constraints in deployed educational pipelines: (i) lightweight defenses that aim to restore instruction boundaries and (ii) limited attack budgets that restrict how many injection candidates can be tried. Figure 6 summarizes results and Table 6 reports mean ASR under defenses compared to the strongest baseline (CoT Hijacking). The goal is not to design new defenses, but to test whether our method remains systematically stronger under common mitigations. Without defenses, our mean ASR is 0.78 versus 0.58 for CoT Hijacking. Input sanitization reduces ASR but preserves a large gap (0.69 vs. 0.47), suggesting our attacks rely on semantic coupling to rubric-driven reasoning rather than brittle token artifacts. Role separation further lowers ASR to 0.65 (baseline 0.44) but does not eliminate attacks, likely because graders must still interpret student text as evidence and often reward structured explanations that can carry subtle control signals. Rubric hardening reduces ASR to 0.62 (baseline 0.41); however, rubric mirroring can remain effective even when output structure is constrained. Output format guards reduce ASR to 0.60, indicating that schema constraints limit syntactic deviation but do not prevent semantic grade inflation. We also evaluate budget sensitivity (Fig. 6b). ASR rises from 0.56 at budget=1 to 0.76 at budget=6 and then saturates near 0.79, implying attackers do not require large query budgets. Overall, defenses provide partial mitigation but leave residual success, supporting the claim that educational vulnerabilities are structural and require semantic verification beyond surface filtering.

Cross-model transfer study

We further test whether the proposed attacks transfer across instruction-tuned models, which is critical because educational platforms may frequently change model backends. Figure 7 and Table 7 show that our method remains consistently strong across three models, achieving ASR 0.77, 0.74, and 0.71. The strongest baseline, CoT Hijacking, drops from 0.61 to 0.52. This implies that our method relies less on idiosyncratic prompt parsing behaviors and more on general instruction-following tendencies shared across models, especially the tendency to treat fluent rubric-like reasoning as evaluative evidence. In contrast, jailbreak and red-team prompts show larger drops, reflecting their dependence on model-specific safety tuning and role-play behaviors. The transfer margin suggests that mitigating our method requires structural changes in educational prompt design or evaluation logic rather than model substitution.

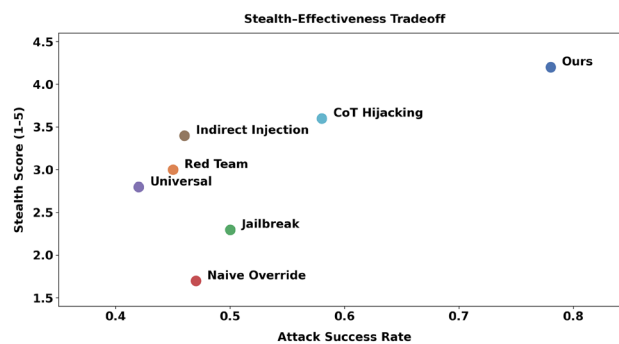


Fig. 8. Stealth–effectiveness tradeoff. Each point is a method with its mean ASR and SS over datasets.

Stealth–effectiveness tradeoff study

This study examines whether higher effectiveness necessarily requires lower stealth. Figure 8 plots the mean ASR against mean SS for our method and baselines. The naive override attack exhibits low stealth (SS 1.7) with only moderate ASR (0.47), indicating that explicit instruction conflicts are both easy to detect and not reliably successful. Jailbreak and red-team prompts occupy the middle region, with SS around 2.3–3.0 and ASR around 0.45–0.50, reflecting that they can sometimes manipulate behavior but often introduce unnatural discourse. CoT Hijacking achieves better balance (ASR 0.58, SS 3.6) because reasoning traces are natural in educational answers. Our method dominates the Pareto frontier at ASR 0.78 and SS 4.2, demonstrating that rubric-aligned embedded injection can be simultaneously more effective and more plausible than competing methods. This result supports the core claim that educational prompts provide a unique channel where adversarial control can be framed as pedagogical reasoning, allowing attacks to avoid the typical tradeoff observed in generic jailbreak settings.

Defense design and evaluation

Educational LLM systems for grading and tutoring are vulnerable to prompt injection because the student submission is simultaneously (i) the *evidence* to be evaluated and (ii) an *instructional channel* that can carry adversarial directives. This dual role is amplified by educational prompt styles that encourage reflective explanations, rubric-aligned self-assessment, and structured reasoning–language patterns that can be repurposed to blur the boundary between “data” and “instructions.” To address this structural issue, we focus on defenses that enforce stronger separation and verification, rather than relying only on keyword filtering. We introduce three practical defenses tailored to rubric-based educational pipelines.

(1) Evidence-First Scoring (EFS). EFS decomposes grading into two explicit stages: **evidence extraction** followed by **scoring**. In the first stage, the grader must extract minimal, criterion-specific evidence spans from the student response for each rubric dimension (e.g., correctness, completeness, explanation quality). The output is a structured record:

$$\{\text{criterion}_k : (\text{evidence_span}_k, \text{justification}_k)\}_{k=1}^K.$$

In the second stage, the grader is instructed to assign scores *only* based on these extracted evidence fields (and the rubric), not on the full student text. This reduces the effective attack surface because injected directives that are not selected as evidence are less likely to influence scoring. EFS also improves auditability: instructors can directly inspect which parts of the student answer were used to justify each awarded point.

(2) Dual-Model Verifier (DMV). DMV adds an independent verifier model that checks whether the grader’s scores are rubric-consistent and evidence-grounded. The verifier receives the rubric, the student answer, the grader’s extracted evidence, and the proposed scores, and outputs ACCEPT/REJECT plus a short rationale. We reject or flag outputs when (i) high scores lack supporting evidence, (ii) extracted evidence does not semantically entail the justification, or (iii) the grader’s rationale appears to treat student meta-instructions as rubric rules. DMV mitigates single-model “instruction-role confusion” by introducing redundancy and an explicit gate before grades are finalized.

(3) Constrained Scoring + Audit (CSA). CSA combines structured output constraints with lightweight semantic auditing. First, the grader is constrained to emit a valid schema (e.g., JSON with fixed keys and bounded score ranges), preventing format manipulation. Second, we apply post-hoc audit rules that enforce educational invariants: each rubric dimension must include an evidence span; justifications must reference extracted spans; and awarding near-maximum scores without adequate evidence triggers a downgrade or manual-review flag. Unlike pure format guards, CSA targets the integrity of the score-evidence linkage and is transparent for deployment (audits are deterministic and easily logged).

Discussion

The additional studies reinforce two broader implications for educational LLM deployments. First, vulnerability is not an edge-case artifact of particular datasets, but a systematic consequence of how educational prompts merge multiple instruction sources, including system policies, rubrics, and student text, into a single context window. The ablation results show that the strongest gains come from rubric alignment and embedded composition,

which together exploit the scoring logic rather than merely bypassing safety constraints. This matters because many practical mitigations focus on banning overt instruction patterns, yet our attacks remain plausible and effective even when sanitization and formatting constraints are applied. Second, robustness experiments highlight that defenses which only constrain output syntax or sanitize inputs reduce success rates but do not eliminate manipulation. The persistence under role separation and rubric hardening suggests that graders must treat student-provided reasoning as potentially adversarial and should adopt verification mechanisms that check whether the generated rationale is grounded in the actual student content and rubric rules. Finally, the tradeoff analysis indicates that stealth is a first-class risk factor in education: attacks that are conspicuous are less realistic, but stealthy attacks can silently alter grades and instructional guidance with high success. From an operational perspective, these findings imply that evaluation of educational LLM systems should include adversarial testing as a standard requirement, similar to reliability and fairness testing, especially in higher-education and vocational contexts where assessment outcomes can affect certification and progression.

Ethics, harm mitigation, and responsible disclosure

This work studies integrity-impacting attacks on educational grading systems. To mitigate harm, (i) all experiments are conducted offline on public benchmarks and local grading pipelines; (ii) we do not target deployed platforms and no real student grades are affected; (iii) we do not release verbatim attack payloads or automation scripts, and instead provide non-weaponizable structural templates; (iv) we include defenses and verification-oriented design recommendations for builders.

Responsible disclosure protocol. When vulnerabilities are found in real deployments, we recommend contacting platform security channels with: a minimal reproduction description, impacted rubric dimensions, and suggested mitigations (evidence-first scoring, verifier checks, constrained scoring). We recommend a remediation window before public release when a specific platform is implicated.

Conclusion

This paper presents a systematic study of prompt injection attacks against educational large language model systems, with an emphasis on high-stakes workflows such as automated grading, rubric-based feedback, and instructional assistance in higher education and vocational training. We propose a structured attack framework that decomposes educational prompts, constructs role-consistent attack vectors, embeds stealthy payloads into student responses, and adapts the injection to rubric language and reasoning conventions. Extensive evaluations across multiple educational datasets and baselines demonstrate that our method achieves consistently higher attack success rates while maintaining strong stealth, and remains effective under common defensive heuristics and realistic attack-budget constraints. Ablation studies confirm that rubric alignment and embedded composition are key drivers of both effectiveness and plausibility, while robustness and transfer analyses highlight that the vulnerability is structural rather than model-specific. These findings suggest that securing educational LLM deployments requires security-aware prompt design and verification mechanisms that constrain how student-controlled text influences scoring and guidance. Future work will explore defense-centric training and runtime detection tailored to educational prompts, as well as stronger guarantees for grading integrity and privacy.

Data availability

The datasets generated and/or analysed during the current study are not publicly available but are available from the corresponding author on reasonable request.

Received: 5 January 2026; Accepted: 26 March 2026

Published online: 31 March 2026

References

1. Brown, T. B. et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems* **33**, 1877–1901 (2020).
2. Kasneci, E. et al. Chatgpt for good? On opportunities and challenges of large language models for education. *Learn. Individ. Differ.* **103**, 102274 (2023).
3. Zhou, C., Li, F., Zhang, J., Wu, X. & Wang, Z. Large language models in education: A survey. *IEEE Transactions on Learning Technologies* (2023).
4. Willison, S. Prompt injection attacks against text-to-image generation. Online Article (2022).
5. Liu, Y., Chen, X. & Zhao, W. Trustworthy ai in education: Challenges and opportunities. *IEEE Intelligent Systems* (2023).
6. Perez, E., Ribeiro, M. T., Yao, S., Zhao, L. & Bansal, G. Red teaming language models with language models. *arXiv* 2202.03286 (2022).
7. Wei, J., Haghtalab, N. & Steinhardt, J. Jailbroken: How does llm safety training fail?. *arXiv* <https://doi.org/10.52202/075280-3508> (2023).
8. Zou, A., Wang, Z., Kolter, J. Z. & Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv* 2307.15043 (2023).
9. Ganguli, D., Askell, A., Schiefer, N., Liao, T. & Chen, M. Red teaming language models to reduce harms. *arXiv* 2209.07858 (2022).
10. OpenAI: GPT-4 Technical Report
11. Biggio, B. & Roli, F. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition* **84**, 317–331 (2018).
12. Goodfellow, I. J., Shlens, J. & Szegedy, C. Explaining and harnessing adversarial examples. *International Conference on Learning Representations* (2015).
13. Liu, J., Zhang, Y., Chen, T. & Wang, H. Prompt injection attacks and defenses in large language models: A survey. *arXiv* 2402.13464 (2024).
14. Kang, D., Li, X. & Stoica, I. Exploiting programmatic weakness of large language models via prompt injection. *arXiv* 2302.12173 (2023).
15. Tang, Z. et al. Rf domain backdoor attack on signal classification via stealthy trigger. *IEEE Trans. Mob. Comput.* **23**(12), 11765–11780 (2024).

16. Greshake, K., et al. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security* (2023)
17. Ke, Z. & Ng, V. Examining the performance of large language models in automated short answer grading. *Proceedings of the AAAI Conference on Artificial Intelligence* (2023)
18. Gilson, A., Safraneck, C., Huang, T., Socrates, V. & Chi, L. How well does chatgpt do on medical licensing exams?. *JMIR Med. Educ.* **9**, 45312 (2023).
19. Chen, H., Xu, J. & Lin, Q. Security risks of large language models in education systems. *IEEE Access* (2024)
20. Triantafyllou, S. A. Understanding ai and google's broader efforts in ai In (ed. Farhaoui, Y.) (2025).
21. Triantafyllou, S. A. Understanding and designing turing machines with innovative applications to computing. In: (ed Farhaoui, Y.) *Artificial Intelligence, Big Data, IOT and Block Chain in Healthcare: From Concepts to Applications*. Information Systems Engineering and Management, **5**, (Springer, Cham, 2024). https://doi.org/10.1007/978-3-031-65014-7_19
22. Lockwood, J. Handbook of automated essay evaluation current applications and new directions mark d. shermis and jill burstein (eds.) (2013). *Writing & Pedagogy* **6**(2), 437–442 (2014).
23. Dzikovska, M. O., et al. Semeval-2013 task 7: The joint student response analysis and 8th recognizing textual entailment challenge. In: *Proceedings of the Seventh International Workshop on Semantic Evaluation* (Association for Computational Linguistics, 2013).
24. Xu, B. et al. Edubench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios. *arXiv* 2505.16160 (2025).
25. Hendrycks, D., Burns, C., Basart, S. & Zou, A. Measuring massive multitask language understanding. In: *International Conference on Learning Representations* (2021)
26. Wang, X., Wei, J. & Schuurmans, D. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations* (2022)

Author contributions

Yunfu Cai led the conceptualization of the study, designed the framework.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Y.C.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026, corrected publication 2026